

MEDICAL DOCUMENT VERSIONING & QA TRACEABILITY API

Engineering Internship Assessment Project Submission Document

Candidate Details & Submission Info

Applicant: JAYANTH REDDY

Role: AI Engineering Intern Candidate

Organization: Tri9T AI

Date: July 2026

Repository URL: <https://github.com/jayanthreddy1507/medical-doc-versioning-api.git>

Documentation Base: docs/approach.md

1. Executive Summary & Setup

The project is a complete production-grade FastAPI application engineered to tackle requirements versioning, traceability, and impact analysis in medical device development. Using an layout-aware parsing pipeline, the system extracts hierarchy trees from blood pressure monitor user manuals, persists them in a versioned relational database, enables named selections, generates QA test-case ideas via LLM integration, and implements active staleness detection.

How to Setup & Run Server:

1. Create and activate Python virtual environment:
`python -m venv venv`
`venv\Scripts\activate` (Windows) or `source venv/bin/activate` (Mac/Linux)
2. Install all required dependencies:
`pip install -r requirements.txt`
3. Run the development server:
`uvicorn app.main:app --reload`
4. Interactive OpenAPI docs will be live at: <http://localhost:8000/docs>

2. Triggering the Ingestion & Revision Flow

We provide a native Python script 'demo_flow.py' that walks through the entire lifecycle.

To execute the E2E verification script:

1. Ensure the FastAPI server is running in a background terminal on port 8000.
2. Run the command in a second terminal:
`.\venv\Scripts\python.exe demo_flow.py`

Steps executed by the verification script:

- Uploads ct200_manual_v1.pdf as 'ct200_manual.pdf' (registers as Document 1, Version 1).
- Locates Node ID for Section 2.1.1.1 (Battery Life) which defines 300 cycles in V1.
- Creates a Selection containing this node.
- Invokes the LLM generator to create QA test cases (marked as `is_stale = False`).
- Re-ingests ct200_manual_v2.pdf under the same filename 'ct200_manual.pdf' (registers as Version 2).
- Resolves structural difference, returning unified diffs showing battery cycles reduced to 250.
- Re-retrieves selection test cases which are now flagged as `is_stale = True` due to the modification.
- Fetches generations by V2 Node ID, proving successful cross-version requirement tracing.

3. Repository Architecture & Layout

The project is structured logically to separate layers of concern:

- app/models/: db_models.py (relational persistence) and document.py (in-memory parsing tree).
- app/schemas/: Pydantic validation models for input request and structured output serialization.
- app/services/parser.py: Custom layout-aware PDF parser utilizing font size/bold metadata.
- app/services/ingestion.py: Ingestion database coordinator and browse logic helper.
- app/services/versioning.py: Path-based matching algorithm and history tracing engine.
- app/services/generation.py: LLM client wrapper orchestrating structured output retries.
- tests/: Complete automated test suite containing 47 unit and integration tests (pytest).

4. PDF Ingestion & Irregularity Handling

Our PyMuPDF parser extracts heading level hierarchies by inspecting per-span fonts. Standard headings are distinguished by bold names ("Helvetica-Bold") and sizes from 10.5pt to 15pt, while body paragraphs use regular weights at 10.0pt. Real-world manual inconsistencies are solved as follows:

- Out-of-Order Sections (e.g. section 3.4 appearing before 3.3 on page 2):
The parser preserves visual reading order in the database stack, preventing structural re-ordering and registering an OUT_OF_ORDER irregularity log.
- Skipped Heading Levels (e.g. jumping from 2.1 to 2.1.1.1, skipping level 3):
The parser links the level-4 node under its nearest existing ancestor (2.1), preserving its true level (4) and logging a SKIPPED_LEVEL warning.
- Duplicate Section Numbers (e.g. two sections labeled '4.2'):
To prevent overwriting, the second occurrence is appended with a suffix '4.2_dup1'. Each node maintains distinct hashes and primary keys, allowing independent versioning and diff tracing.

5. Document Versioning & Matching

When a new manual revision is uploaded, nodes are compared using a Hierarchical Path Matcher:

1. Nodes are matched primarily by absolute path signatures (e.g., '/Document Root/Specifications/General').
2. If path signatures differ (due to parent renaming), Gestalt pattern matching (SequenceMatcher) is triggered on the heading titles under the same hierarchy level with a 70% threshold.
3. Nodes with modified text are flagged as 'modified' and unified line-by-line diffs are created by comparing SHA-256 content hashes.

Matching Failure Modes:

If a high-level chapter is renamed and section numbers are re-ordered simultaneously, path comparison fails. In this edge case, V1 nodes are marked as 'removed' and V2 nodes are treated as 'added', breaking history tracing.

6. LLM Generation & Self-Repair Retry Loop

The system generates 3-5 QA test cases using Pydantic structured output constraints (responseMimeType: application/json).

Self-Repair Correction Loop:

If the LLM generates malformed JSON, or fails Pydantic schema validation, the system catches the parser error, and prompts the LLM again (up to 3 times) appending the validation error message and its previous malformed text. This self-repair loop guarantees stable, structured JSON output in production.

Duplicate Submission Policy:

Repeated requests to generate test cases for the same selection ID return the cached generation immediately. This drastically reduces billing costs and api latency, while ensuring output consistency. Bypassed only when explicitly requested using force_regenerate=true.

7. Active Staleness Detection Limits

Selected nodes are linked to their exact content hashes at generation time. On retrieval, the system resolves the latest version, compares the path match content hash, and flags staleness status ('up_to_date', 'stale', 'removed').

Limitations:

- Formatting changes (e.g. layout or line breaks) change the content hash, causing false stale flags.
- Semantic rephrasings that preserve functional equivalence trigger a stale flag (binary hash mismatch).
- Out-of-scope shifts: a changed constraint outside the selection area goes undetected (false negative).

8. Required Decision Log Answers

Q1: What's the one part of this system most likely to silently give wrong results without erroring? How would you

Answer: The heading level classifier during PDF extraction. If a heading in a document uses non-standard bold flags, or slightly modified font sizes (e.g., 10.2pt instead of 10.5pt), the parser will treat the heading as body text and merge it into the preceding section's text. The tree builds successfully without throwing any errors, but the hierarchy is broken. How to catch it: Post-parsing validation checks. We scan extracted body paragraphs using regex. If a paragraph starts with a subsection number prefix (e.g. `^\d+(\.\d+)+\s+[A-Z]`), we flag it as a parsing anomaly warning.

Q2: Where did you choose simplicity over correctness because of time, and what would break first if this went to

Answer: The custom layout-based table extractor. It relies on grouping spans strictly by identical y-coordinates with a small vertical tolerance.

What would break first: If a table has cell blocks containing multi-line wrapped text or merged vertical headers, the extractor will treat each line of text as a separate row or merge columns incorrectly. In production, tables containing wrapped cells will be stored as garbled data rows, losing cell alignment and destroying traceability.

Q3: Name one input (to your parser, your versioning matcher, or your LLM call) that you did not handle, and what

Answer: A scanned/image-only PDF document containing no embedded text spans.

What the system does: PyMuPDF's `page.get_text('dict')` returns an empty span list. The parser processes this as a zero-text document, creating a tree with only a root node, and returns a success status of 201 (with no children nodes and empty content). In production, this must be caught by checking if the page contains image blocks but no text, triggering an OCR pipeline fallback (e.g., Tesseract).

9. Project Submission Email Template

To: careers@tri9t.ai

Subject: AI Engineering Intern Submission - JAYANTH REDDY

Dear Tri9T AI Hiring Team,

I am pleased to submit my entry for the AI Engineering Internship Assessment project. I have built a robust document parsing, versioning, and test-case generation backend satisfying all core requirements, including the edge cases present in the CT-200 manual.

Key Project Links:

- GitHub Repository URL:

<https://github.com/jayanthreddy1507/medical-doc-versioning-api.git>

- Technical Approach Document:

<https://github.com/jayanthreddy1507/medical-doc-versioning-api/blob/main/docs/approach.md>

Features Implemented:

1. PyMuPDF layout-aware tree parser handling out-of-order, duplicates, and skipped levels.
2. Hierarchical path-based & fuzzy SequenceMatcher version matching engine with unified diffs.
3. Named, version-pinned Selections API.
4. LLM QA generator with Pydantic JSON validation and self-repair loops (retries with error logs).
5. SQLite database using native JSON extensions as our light NoSQL test-cases store.
6. Active staleness/impact detection on selection retrievals and cross-version node tracing.
7. Complete E2E verification script (demo_flow.py) validating the versioning lifecycle.
8. Fully documented Decision Log answers addressing parser edge-cases and limitations.

Thank you for this opportunity, and I look forward to walking you through my implementation and design choices!

Best regards,

JAYANTH REDDY

jayanthreddy1507@gmail.com